

README

User Guide — Wiring CLI Coding Agents to Helix

Field	Value
Revision	1
Created	2026-09-05
Last modified	2026-09-05
Status	DRAFT — describes the intended, evidence-backed wiring; confirm against your own run before relying on it
Status summary	Documents how opencode, pi, crush, and Claude Code point at Helix’s local LLM surfaces, how to verify each one yourself, and which surface to pick when tool-calling matters.
Scope	Operator-facing usage of the four Helix LLM surfaces from four CLI coding agents on this host
Authority	Facts here are either the output of live probes run 2026-09-05 (see docs/research/cli_agent_config_schemas/EVIDENCE.md) or explicit vendor documentation, cited inline. Nothing here is asserted as “your current config” — verify with the commands given.

Anti-bluff note. This guide was written from probe evidence and shipped-binary/vendor-doc research captured on 2026-09-05, not from having watched an installer run end to end. Wherever a claim depends on an installer having already wired your local config, it is phrased as “after wiring, you should see...” — treat any mismatch between that

and your own `crush models / opencode models / pi --list-models` output as the thing to trust, not this document.

Table of contents

- 1. [What gets wired, and where](#)
 - 2. [The four Helix surfaces](#)
 - 3. [Choosing a surface: the one thing that will surprise you](#)
 - 4. [Per-agent configuration](#)
 - 5. [TLS for the self-signed gateway](#)
 - 6. [Verifying each agent yourself](#)
 - 7. [Undoing / removing the Helix providers](#)
 - 8. [Re-running the installer: idempotency](#)
 - 9. [The context-size cautionary tale](#)
 - 10. [Where to go next](#)
-

1. What gets wired, and where

Each supported CLI agent reads its own provider configuration from its own file, in its own format. Wiring Helix into an agent means adding one (or more) **custom OpenAI-compatible provider entries** pointing at one of the Helix LLM surfaces described in [§2](#), plus the model id(s) that surface serves.

There is no single shared config file — four agents, four files, four shapes:

Agent	Version probed	Config file	Format
opencode	1.18.29	~/.config/ opencode/ opencode.json	JSON
pi	0.84.4	~/.pi/agent/ models.json	JSON
crush	0.91.2	~/.config/ crush/crushrc	Bash directives (not JSON)
claude (Claude Code)	2.1.261	—	N/A for local OpenAI-compatible providers; see §4.4

If an installer has run against your machine, you should find Helix entries already present in the files above for `opencode`, `pi`, and `crush`. If it has not run yet, [§4](#) below shows you exactly what to add and how to add it by hand.

2. The four Helix surfaces

Helix exposes four distinct HTTP surfaces on this host. They are not interchangeable — same underlying model in two cases, different capability in each:

Surface	URL	Auth	Model id(s) served	Tool-calling
Coder (raw llama.cpp)	http://127.0.0.1:18434/v1	none	qwen2.5-coder-3b-instruct-q4_k_m	emits a fenced JSON block with the message text, not a structured <code>tool_calls</code> field
Gateway (HelixLLM)	https://127.0.0.1:8443/v1	none (self-signed TLS)	helixllm-anton-qwen2-5-coder-3b-instruct-q4_k_m-f6771589d190	real <code>tool_calls</code> , <code>finish_reason:"tool_calls"</code>
HelixAgent	http://127.0.0.1:7061/v1	none	helixagent-llm, helixagent-debate, helixagent-ensemble, helix-llm, helix-debate	not probed — treat as unconfirmed until you check yourself
HelixCode	http://127.0.0.1:8080	API key required (401 without one)	/v1/messages (Anthropic-shaped) + /v1/chat/completions	—

All four were reached with a plain `curl` against `/v1/models` (or the equivalent) on 2026-09-05; see `docs/research/cli_agent_config_schemas/EVIDENCE.md` for the raw transcripts. Ports and reachability can change between hosts and

sessions — if a `curl` to one of these fails for you, that surface is down or not yet started, not a documentation error.

3. Choosing a surface: the one thing that will surprise you

Coder and Gateway serve the same underlying model. Point an agent at Coder (:18434) and its tool-calling requests get a chunk of JSON text pasted into the assistant's reply — the agent's tool-call parser will not recognize it, because it never arrives as a structured `tool_calls` field. Point the same agent at Gateway (:8443) instead, and tool-calling works, because the gateway performs response translation the raw llama.cpp server does not do on its own.

In other words: **which surface you point at — not just which model you ask for — decides whether tool-calling works.** If your agent (or your workflow) needs tools/functions, use the **Gateway**, not the Coder surface, even though they can appear to offer the “same” model. If you only need plain chat completion with no tool use, the Coder surface is simpler (no TLS to deal with — see [§5](#)).

HelixAgent (:7061) exposes different model ids again (`helixagent-llm`, `helixagent-debate`, `helixagent-ensemble`, plus `helix-llm/helix-debate` aliases) and was not probed for tool-calling behavior in this pass — do not assume it behaves like either Coder or Gateway until you have checked.

HelixCode (:8080) is a different shape entirely: it speaks the Anthropic Messages API (`/v1/messages`) as well as `/v1/chat/completions`, and it requires an API key. It is covered separately in [§4.4](#) because, of the four agents this guide covers, only Claude Code has a plausible path to it — and that path was, at the time of writing, still under verification by a separate work stream. Do not assume it works; check the status note in that section before relying on it.

4. Per-agent configuration

4.1 opencode (1.18.29)

Config file: `~/.config/opencode/opencode.json`.

Add a provider entry under `provider.<your-id>`, using the `@ai-sdk/openai-compatible` npm adapter, pointing `options.baseUrl` at one of the surfaces from §2, and enumerating the model id(s) that surface serves:

```
{  
  "provider": {
```

```

    "helixgw": {
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseUrl": "https://127.0.0.1:8443/v1"
      },
      "models": {
        "helixllm-anton-qwen2-5-coder-3b-instruct-q4_k_m-
        f6771589d190": {}
      }
    }
  }
}

```

Notes:

- Omit `apiKey` entirely for these keyless local surfaces (Coder, Gateway, HelixAgent). Do not supply a placeholder value — the adapter treats an omitted key correctly for a server that does not check one.
- `opencode` **restarts required**: it reads this file at process start, so after adding or changing an entry, quit and relaunch `opencode` before it is visible.
- The **first use of a newly-declared `npm:-backed provider`** triggers `opencode` to download that npm package (and its dependencies) into its config directory — roughly 63 MB of `node_modules/` was observed for the adapter package plus its ecosystem. This needs network access the first time; subsequent runs reuse what was downloaded.
- Multiple config files under `~/.config/opencode/` (`config.json`, `opencode.json`, `opencode.jsonc`, and a TOML config) are all read and merged, later file wins. If you already have entries in more than one of these, know that they combine rather than one silently overriding the others outright — check all of them if something looks wrong.

4.2 pi (0.84.4)

Config file: `~/.pi/agent/models.json` (overridable via the `PI_CODING_AGENT_DIR` environment variable, which changes pi's whole config directory, not just this file).

Add a provider under `providers.<your-id>`:

```

{
  "providers": {
    "helixgw": {
      "baseUrl": "https://127.0.0.1:8443/v1",
      "api": "openai-completions",
      "apiKey": "local"
    }
  }
}

```

```
}  
}
```

Notes:

- Unlike the other three agents, **pi requires a non-empty apiKey value even for a keyless server** — the field cannot be omitted. Any placeholder string (e.g. "local") works; the value is not actually checked by these local surfaces.
- No restart is required: pi reloads provider config when you run its `/model` command inside a session.
- pi ships its own documentation under its installed package's `docs/` directory (`models.md`, `custom-provider.md`, `providers.md`, `settings.md`, `environment-variables.md`) — worth reading directly if you need an option not shown here.

4.3 crush (0.91.2)

Config file: `~/.config/crush/crushrc` — this is **Bash**, not JSON. Provider registration is a command line inside that file, or run directly against a running crush:

```
provider add helixgw --type openai-compatible --base-url https://  
127.0.0.1:8443/v1 --api-key local --discover-models true
```

Notes:

- With `--discover-models true`, crush queries the surface's `/v1/models` endpoint itself and populates the model list — you do not need to hand-declare model ids the way `opencode` and `pi` require. A bare `provider add` with no `model add` calls was enough, in testing, to make the served model visible in `crush models`.
- crush **restart required** after editing `crushrc` directly; re-running `provider add` against a live crush merges into the existing entry rather than duplicating it.
- TLS trust for the self-signed Gateway goes through the Go standard library's `SSL_CERT_FILE` environment variable — there is no TLS-related field anywhere in crush's own config schema (crush schema was inspected directly; no `tls/insecure/verify/cert/proxy` key exists). See [§5](#).
- **crush writes a .crush/ data directory into your current working directory by default.** If you run crush from inside this repository (or any project directory) without overriding this, it will create `.crush/` there. Pass `--cwd <dir>` and/or `--data-dir <dir>` to control where that lands, or see the FAQ entry in [FAQ.md](#) for how to avoid it entirely.

4.4 Claude Code (2.1.261)

Claude Code cannot accept a custom OpenAI-compatible provider. This is a vendor constraint, not a Helix limitation: Anthropic’s own documentation for the LLM Gateway Protocol lists exactly three client-selectable API formats for Claude Code — Anthropic Messages, Amazon Bedrock InvokeModel, and Google Agent Platform `rawPredict` — with no OpenAI Chat Completions option at all, and states explicitly that “Anthropic doesn’t endorse, maintain, or audit third-party gateway products, and doesn’t support routing Claude Code to non-Claude models through any gateway.” Claude Code’s optional model-discovery feature also only keeps model ids that contain `claude` or `anthropic` (case-insensitively) anywhere in the string — a Helix model id would be filtered out even if discovery were pointed at a Helix surface.

The only surface with a plausible path into Claude Code is **HelixCode’s Anthropic-shaped `/v1/messages` endpoint on `:8080`** (via `ANTHROPIC_BASE_URL` plus an API key, since `:8080` returns 401 without one). **This path was, at the time this guide was written, still under verification by a separate work stream — do not assume it works.** Check that stream’s findings (or re-run the verification yourself against `:8080`) before relying on Claude Code against Helix for anything.

If you need Claude Code today, treat `opencode`, `pi`, or `crush` — pointed at Gateway or Coder — as the supported path for local Helix models.

5. TLS for the self-signed gateway

The Gateway surface (`https://127.0.0.1:8443/v1`) serves a self-signed TLS certificate. A plain HTTPS client will refuse it by default.

- `curl` needs `--cacert <path-to-cert>`, or it fails with `curl: (60) SSL certificate problem: self-signed certificate`.
- `crush` (Go binary) trusts the certificate when `SSL_CERT_FILE=<path-to-cert>` is set in its environment before it runs. This was confirmed directly: without it, `crush models` returns zero Gateway model entries (discovery silently fails); with it set, the Gateway’s model id appears.
- `opencode` and Claude Code (both Node/Bun-based) recognize `NODE_EXTRA_CA_CERTS` and `NODE_TLS_REJECT_UNAUTHORIZED` as the equivalent knobs — these string literals are present in both shipped binaries, though a full end-to-end proof against the Gateway specifically was inconclusive for `opencode` (the test run hit a 280-second timeout with no output; see `docs/research/cli_agent_config_schemas/EVIDENCE.md` for the raw record — that is an honest “no verdict either way,” not a documented failure).

- `pi` recognizes `SSL_CERT_FILE` in principle but a live end-to-end run against the Gateway also timed out (90 seconds) both with and without `NODE_EXTRA_CA_CERTS` set — again, no usable verdict either way was obtained during this probing pass.

Practical takeaway: if you plan to use the Gateway surface, obtain its certificate and export `SSL_CERT_FILE` (or your agent's equivalent) before launching the agent. If model discovery for the Gateway silently returns nothing, TLS trust — not the provider config — is the first thing to check.

6. Verifying each agent yourself

Do not trust that wiring succeeded just because a file was written. Run the agent's own model-listing command and `grep` for `helix`:

```
# opencode
```

```
opencode models | grep -i helix
```

```
# pi
```

```
pi --list-models | grep -i helix
```

```
# crush
```

```
crush models | grep -i helix
```

```
# Claude Code – N/A for local OpenAI-compatible surfaces (see §4.4);
```

```
# for HelixCode's Anthropic-shaped surface, check with that stream's own
```

```
# verification rather than this command.
```

A model id appearing in that output means the provider is *registered* — it does **not** by itself mean a real request will succeed. See §9 for why registration and usability are two different things, and confirm with a real, small completion request before trusting the wiring for real work:

```
crush run -q "Reply with exactly the word: HELIXOK"
```

(An equivalent one-shot invocation exists for `pi` — `pi -p --mode json --provider <id> --model <id> --api-key local "Reply with exactly the word: HELIXOK"` — and for `opencode` via its non-interactive run mode. Consult each agent's own `--help` for the exact current flag names, since these can change between releases.)

7. Undoing / removing the Helix providers

Because each agent's Helix entry is just a provider block in that agent's own config file, removing Helix from an agent means removing that block and nothing else:

- **opencode:** delete the `provider.<your-id>` object from `~/.config/opencode/opencode.json` (or whichever of the merged config files it lives in — see the note in [§4.1](#)), then restart opencode.
- **pi:** delete the `providers.<your-id>` object from `~/.pi/agent/models.json`. No restart needed.
- **crush:** either delete the corresponding `provider add ...` line from `crushrc`, or run `crush model remove <provider>/<id>` for each declared model followed by removing the provider line, then restart crush.
- **Claude Code:** nothing to undo for the local OpenAI-compatible surfaces, since it never accepted them in the first place ([§4.4](#)). If a separate stream wired `ANTHROPIC_BASE_URL` / an API key for the HelixCode surface, undo whatever mechanism that stream used to set those (environment variable, shell profile line, or settings file — check its own documentation).

None of this touches the Helix surfaces themselves (Coder, Gateway, HelixAgent, HelixCode keep running); it only removes an agent's knowledge of them.

8. Re-running the installer: idempotency

If an automated installer wires these providers for you, re-running it against an agent that already has a Helix entry should **merge into the existing config rather than duplicate or clobber it** — this is the expected, intended behavior for this kind of provider-registration step, consistent with how each agent's own tooling behaves when you register the same provider twice by hand (`crush provider add` on an existing id merges; re-declaring a model in `opencode.json` or `pi's models.json` simply overwrites that one entry, leaving everything else in the file untouched).

Concretely, a safe re-run should leave:

- any provider entries you added yourself, for services unrelated to Helix, completely untouched;
- the Helix entries either unchanged (if nothing changed on the Helix side) or updated in place (e.g. a new model id, a changed port) — never appended as a second, differently-named duplicate of the same surface.

Confirm this for your own installer rather than trusting this paragraph: run it twice in a row and diff the config file (or re-run the `grep -i helix` checks from [§6](#) before and after) to see for yourself that nothing duplicated.

9. The context-size cautionary tale

Registering a model successfully does not mean it is usable. On 2026-09-05, with the Coder surface's context window (`n_ctx`) set to 4096 tokens, a real pi request against it was rejected outright:

```
400: {"code":400,"message":"request (141440 tokens) exceeds the available context size (4096 tokens), try increasing it","n_prompt_tokens":141440,"n
```

The agent's own system prompt alone — before any user turn is even added — was over 141,000 tokens once tokenized against this model, thirty-four times the 4096-token window. The provider was registered correctly, the request reached the server correctly, and the server replied correctly — it just could not do anything useful with an agent whose baseline prompt does not fit.

The Coder surface's context window has since been raised (to 32768 tokens as of 2026-09-05), which should leave considerably more headroom, but the lesson generalizes: **a model appearing in <agent> models proves the plumbing works — it proves nothing about whether a real turn will fit.** If an agent registers cleanly but every real request fails, check the context window of the surface you pointed it at before assuming the wiring is broken. See [FAQ.md](#) for the specific “lists the model but every request fails” question.

10. Where to go next

- [FAQ.md](#) — targeted answers to the questions this wiring tends to raise.
- [ARCHITECTURE.md](#) — a diagram of all four agents, all four surfaces, and which can reach which.
- `docs/research/cli_agent_config_schemas/EVIDENCE.md` — the raw, dated probe transcripts every factual claim in this guide traces back to.